Seminararbeit SYCL

Zwischenstand und bisherige Erkenntnisse

Erlind Sejdiu (106357) · Stand: 22. 10. 2026

Inhaltsverzeichnis

1 Worum geht es? 1

2 Die wichtigsten Begriffe in einfachen Worten 2

3 Vorgehen 2

4 Erkenntnisse 3

4.1 Derselbe Code läuft auf CPU und GPU . . . 3
4.2 Der Einstieg ist klein, der Effekt groß . . . 3
4.3 Der erste Lauf ist deutlich langsamer . . . 4
4.4 Bei kleinen Aufgaben dominiert der Datentransport . . . 4
4.5 Automatische und manuelle Speicherverwaltung sind gleich schnell . . . 4
4.6 Hardwarenahe Optimierung wirkt je nach Gerät sehr unterschiedlich . . . 4
4.7 Portabilität ist nicht gleich Performance-Portabilität . . . 5
4.8 Praktische Hürden . . . 6

5 Grenzen der bisherigen Messungen 6

6 Nächste Schritte und Fragen für das Gespräch 6

1 Worum geht es?

Moderne Rechner enthalten neben der CPU oft weitere Recheneinheiten, vor allem Grafikkarten (GPUs). Eine GPU kann sehr viele gleichartige Rechnungen gleichzeitig ausführen und ist dadurch bei passenden Aufgaben um Größenordnungen schneller als eine CPU.

Das Problem: Jeder Hersteller hat seine eigene Programmierschnittstelle. Code für NVIDIA-GPUs wird typischerweise in CUDA geschrieben und läuft nur auf NVIDIA-Hardware. Für AMD- oder Intel-GPUs oder für die CPU müsste man das Programm neu schreiben.

SYCL (gesprochen „sickle“) ist ein offener Standard der Khronos Group, die auch OpenGL und Vulkan verantwortet. Die Idee: Man schreibt *einmal* normales C++, und derselbe Quellcode läuft auf CPUs und auf GPUs verschiedener Hersteller. Es gibt mehrere Compiler, die SYCL umsetzen, u. a. *DPC++* von Intel und *AdaptiveCpp* (Universität Heidelberg).

Die Leitfrage meiner Arbeit ist:

> *Wie aufwendig ist es, ein bestehendes C++-Programm auf SYCL umzustellen, was bringt es an Leistung, und hält das Versprechen „einmal schreiben, überall ausführen“?*

2 Die wichtigsten Begriffe in einfachen Worten

Host und Gerät (Device)

Der *Host* ist die CPU, auf der das Hauptprogramm läuft. Ein *Gerät* ist die Hardware, auf der die eigentliche Rechnung läuft, z. B. eine GPU. Auch die CPU selbst kann als Gerät dienen.

Queue (Warteschlange)

Über eine Queue schickt der Host Arbeitsaufträge an genau ein Gerät. Welches Gerät das ist, kann man beim Programmstart über eine Umgebungsvariable festlegen, ohne den Code zu ändern.

Kernel

Das Stück Code, das auf dem Gerät ausgeführt wird. In SYCL ist das eine gewöhnliche C++-Funktion (Lambda).

Work-Item

Eine einzelne Ausführung des Kernels, vergleichbar mit einem sehr leichtgewichtigen Thread. Bei der Matrixmultiplikation berechnet ein Work-Item genau einen Eintrag der Ergebnismatrix. Bei einer 2048×2048-Matrix sind das rund 4 Millionen Work-Items.

Work-Group

Eine Gruppe von Work-Items, die zusammenarbeiten, sich einen kleinen, schnellen Zwischenspeicher teilen (*Local Memory*) und aufeinander warten können (*Barriere*).

Unified Shared Memory (USM)

Die Art, wie Speicher zwischen Host und Gerät geteilt wird. Entweder *shared* (SYCL verschiebt die Daten automatisch) oder *device* (der Programmierer kopiert die Daten selbst).

Backend

Die herstellerspezifische Schicht, auf die SYCL intern zurückgreift, z. B. CUDA für NVIDIA oder OpenCL für die CPU. Der Programmierer sieht sie nicht.

GFLOPS

Milliarden Gleitkommarechnungen pro Sekunde, das übliche Maß für Rechenleistung. Höher ist besser.

3 Vorgehen

Als Beispiel dient die Multiplikation zweier quadratischer Matrizen ($C = A \cdot B$). Sie ist leicht zu verstehen, braucht viel Rechenzeit ($2n^3$ Operationen) und ist ein Standardbeispiel für GPU-Programmierung.

Ein einfaches C++-Programm wird in Stufen auf SYCL umgebaut. Jede Stufe führt genau ein neues Konzept ein. Alle Stufen messen auf die gleiche Weise und prüfen ihr Ergebnis gegen eine Referenzrechnung.

Stufe	Was passiert	Neues Konzept
0	Ausgangsprogramm: drei verschachtelte Schleifen auf einem CPU-Kern	–
1	Die äußeren Schleifen werden durch einen parallelen Kernel ersetzt	Queue, Kernel, USM *shared*
2	Daten werden explizit auf das Gerät und zurück kopiert	USM *device*, Kopieren
3	Work-Groups laden Teilblöcke („Kacheln“) in den schnellen Local Memory	Work-Group, Local Memory, Barriere
4	Kein neuer Code: Stufe 1–3 auf CPU und GPU ausführen	Geräteauswahl

Tabelle 1: Die Stufen des Beispielprojekts

Messumgebung. AMD Ryzen 5 5600X (6 Kerne) und NVIDIA GeForce RTX 5060 Ti. Compiler: DPC++ 7.2 (Intel LLVM). Die CPU wird über das OpenCL-Backend angesprochen, die GPU über das CUDA-Backend. Gemessen wird jeweils der Median aus fünf Wiederholungen, der allererste Lauf wird getrennt erfasst.

4 Erkenntnisse

4.1 Derselbe Code läuft auf CPU und GPU

Alle Stufen liefern auf beiden Geräten korrekte Ergebnisse. Um zwischen CPU und GPU zu wechseln, wird keine einzige Codezeile geändert, sondern nur eine Umgebungsvariable beim Start gesetzt.

Eine Einschränkung gibt es: Das Programm muss beim Kompilieren für alle gewünschten Geräte übersetzt werden. Ein nur für NVIDIA übersetztes Programm bricht auf der CPU mit der Meldung ab, es gebe „kein Abbild für das gewählte Gerät“. Das ist ein Compiler-Schalter, keine Codeänderung, aber man muss es wissen.

Kernaussage: *Das Versprechen „einmal schreiben, überall ausführen“ hält im getesteten Rahmen.*

4.2 Der Einstieg ist klein, der Effekt groß

Der Schritt von Stufe 0 zu Stufe 1 betrifft im Wesentlichen die zwei äußeren Schleifen, die durch einen `parallel_for`-Aufruf ersetzt werden. Die eigentliche Rechnung bleibt gleich.

Variante ($n = 1024$)	Zeit	GFLOPS
Stufe 0, ein CPU-Kern	4484 ms	0,5
Stufe 1, CPU (alle Kerne, SYCL)	87 ms	25
Stufe 1, GPU	1,9 ms	1105
Stufe 3, GPU	1,5 ms	1395

Tabelle 2: Rechenzeit ohne Datentransfer

Die GPU ist hier über 2000-mal schneller als das Ausgangsprogramm. Diese Zahl ist aber

mit Vorsicht zu lesen: Stufe 0 ist absichtlich eine einfache Implementierung auf nur einem Kern. Ein Vergleich mit einer optimierten CPU-Bibliothek würde deutlich kleiner ausfallen.

Kernaussage: *Mit wenig Umbauaufwand ist bereits ein Großteil der möglichen GPU-Leistung erreichbar.*

4.3 Der erste Lauf ist deutlich langsamer

Beim ersten Aufruf übersetzt die Laufzeitumgebung den Kernel noch passend für das konkrete Gerät (*Just-in-Time-Kompilierung*) und initialisiert das Gerät. Bei $n = 256$ dauert Stufe 1 auf der GPU im ersten Lauf 0,75 ms statt 0,04 ms (etwa 20-mal länger), auf der CPU 118 ms statt 0,4 ms.

Kernaussage: *Für kurze Programme, die nur einmal rechnen, kann diese Anlaufzeit den Vorteil der GPU aufheben. Bei Messungen muss der erste Lauf getrennt betrachtet werden.*

4.4 Bei kleinen Aufgaben dominiert der Datentransport

Die GPU hat eigenen Speicher. Die Matrizen müssen erst dorthin und das Ergebnis zurück kopiert werden (Stufe 2, GPU):

n	**Rechnung**	**Kopieren**	**Anteil Kopieren**
256	0,04 ms	0,17 ms	82 %
1024	1,97 ms	1,34 ms	40 %
2048	15,87 ms	3,93 ms	20 %

Tabelle 3: Rechnung und Datentransfer auf der GPU

Der Rechenaufwand wächst mit n^3, die Datenmenge nur mit n^2. Je größer die Aufgabe, desto mehr lohnt sich die GPU.

Kernaussage: *Ob sich eine GPU lohnt, hängt von der Problemgröße ab. Bei kleinen Matrizen kostet das Kopieren mehr als die Rechnung.*

4.5 Automatische und manuelle Speicherverwaltung sind gleich schnell

Stufe 1 (SYCL verschiebt die Daten selbst) und Stufe 2 (Daten werden manuell kopiert) sind in der reinen Rechenzeit praktisch gleich schnell, z. B. 1,94 ms gegenüber 1,97 ms auf der GPU bei $n = 1024$. Der Unterschied liegt darin, *wann* die Kopierkosten anfallen: Bei Stufe 1 werden sie beim ersten Zugriff versteckt mitbezahlt, bei Stufe 2 sind sie sichtbar und planbar.

Kernaussage: *Die automatische Variante ist für den Einstieg ausreichend. Die manuelle Variante macht die Kosten sichtbar und kontrollierbar.*

4.6 Hardwarenahe Optimierung wirkt je nach Gerät sehr unterschiedlich

In Stufe 3 arbeiten jeweils 16×16 Work-Items als Gruppe zusammen. Sie laden gemeinsam einen Teilblock der Matrizen in einen kleinen, schnellen Speicher und rechnen von dort weiter. Dadurch muss jeder Wert seltener aus dem langsamen Hauptspeicher gelesen werden.

Gerät	Stufe 2	Stufe 3	Gewinn
GPU, $n = 1024$	1,97 ms	1,54 ms	+28 %
GPU, $n = 2048$	15,87 ms	12,12 ms	+31 %
CPU, $n = 1024$	83,1 ms	18,4 ms	×4,5
CPU, $n = 2048$	450,9 ms	129,2 ms	×3,5

Tabelle 4: Wirkung der Kachel-Optimierung (Stufe 2 → Stufe 3)

Vermutete Erklärung (nicht gesondert gemessen): Die GPU fängt schon in Stufe 2 durch eigene Caches und gebündelte Speicherzugriffe viel ab. Die CPU profitiert dagegen stark davon, dass die Teilblöcke in ihren Cache passen. Bei sehr kleinen Matrizen ($n = 256$) bringt die Optimierung auf der GPU nichts, dort ist Stufe 3 sogar etwas langsamer.

Kernaussage: *Dieselbe Optimierung bringt auf einem Gerät 30 %, auf dem anderen ein Vielfaches.*

4.7 Portabilität ist nicht gleich Performance-Portabilität

Hier liegt die bisher wichtigste Erkenntnis. Man muss zwei Begriffe unterscheiden:

- **Portabilität:** Der Code läuft auf jedem Gerät und rechnet richtig.
- **Performance-Portabilität:** Der Code nutzt auf jedem Gerät auch einen ähnlich großen Teil der möglichen Leistung, ohne gerätespezifische Anpassung.

Um das zu prüfen, wurde Stufe 3 mit verschiedenen Kachelgrößen T gebaut (eine Work-Group umfasst $T \times T$ Work-Items) und auf beiden Geräten gemessen. Die Prozentangabe zeigt, wie nah die Variante an der besten Kachelgröße für *dieses* Gerät liegt:

T	GPU	CPU	PP
8	1135 GFLOPS (83 %)	71 GFLOPS (45 %)	0,59
16	**1368** GFLOPS (100 %)	137 GFLOPS (88 %)	0,93
32	1354 GFLOPS (99 %)	**157** GFLOPS (100 %)	0,99
64	*startet nicht*	142 GFLOPS (91 %)	0

Tabelle 5: Kachelgröße und Leistung, $n = 2048$

Drei Beobachtungen:

1. **Die beste Einstellung unterscheidet sich je nach Gerät:** Auf der GPU ist $T = 16$ am schnellsten, auf der CPU $T = 32$. Bei $n = 1024$ erreicht die CPU mit $T = 16$ nur 74 % ihres besten Werts.
2. **Eine Einstellung kann ein Gerät ganz ausschließen:** Bei $T = 64$ umfasst eine Work-Group 4096 Work-Items. NVIDIA-GPUs erlauben höchstens 1024, der Kernel startet dort also gar nicht. Auf der CPU läuft dieselbe Variante dagegen gut.
3. **Eine schlechte Einstellung trifft die Geräte unterschiedlich stark:** $T = 8$ kostet die GPU 17 %, die CPU 55 %.

Die Spalte **PP** ist eine in der Forschung gängige Kennzahl für Performance-Portabilität (Pennycook, Sewall und Lee, *A Metric for Performance Portability*, 2016). Sie fasst die Prozentwerte aller Geräte zu einem Wert zwischen 0 und 1 zusammen (harmonischer Mittelwert). Ein einziges schwaches Gerät zieht den Wert stark nach unten, und läuft der Code auf einem Gerät gar nicht, ist er 0.

Einschränkung: Die Prozentwerte beziehen sich auf die beste gemessene Kachelgröße, nicht auf die theoretische Höchstleistung der Hardware.

Kernaussage: *SYCL sorgt dafür, dass der Code überall läuft. Damit er überall schnell läuft, müssen Parameter wie die Kachelgröße weiterhin pro Gerät abgestimmt werden.*

4.8 Praktische Hürden

- **Einrichtung der Werkzeuge:** Die Standardinstallation von Intel oneAPI sah auf dem Testrechner nur die CPU. Für die NVIDIA-GPU war eine eigene DPC++-Version mit CUDA-Unterstützung nötig. Der Aufwand steckt also eher in der Einrichtung als im Programmieren.
- **Übersetzen für alle Zielgeräte:** Die Zielgeräte müssen beim Kompilieren angegeben werden (siehe 4.1).
- **Fehlermeldungen:** Fehler im Kernel (z. B. zu große Work-Groups) zeigen sich erst zur Laufzeit und nur auf dem betroffenen Gerät.

5 Grenzen der bisherigen Messungen

- Getestet wurden zwei Geräte (eine CPU, eine NVIDIA-GPU). AMD- und Intel-GPUs fehlen noch.
- Die Messreihen wurden jeweils einmal aufgenommen (mit fünf Wiederholungen pro Messpunkt). Kleine Unterschiede wie 1354 gegenüber 1368 GFLOPS können Messrauschen sein.
- Das Ausgangsprogramm (Stufe 0) ist absichtlich einfach gehalten. Die großen Beschleunigungsfaktoren gegenüber Stufe 0 sind deshalb kein fairer Vergleich mit optimiertem CPU-Code.
- In früheren Messungen (z. B. auf den Vortragsfolien) lief die CPU über AdaptiveCpp mit OpenMP statt über DPC++ mit OpenCL. Die Ergebnisse sind nicht direkt vergleichbar, weil der Compiler einen deutlichen Einfluss hat.

6 Nächste Schritte und Fragen für das Gespräch

1. Messungen mehrfach wiederholen und Streuung angeben.
2. Wenn möglich weitere Hardware testen (AMD- oder Intel-GPU).
3. Vergleich der beiden SYCL-Compiler (DPC++ und AdaptiveCpp) auf derselben Hardware.
4. Optional: Vergleich mit einer optimierten Bibliothek (z. B. cuBLAS auf der GPU) als Obergrenze, um die Prozentwerte auf die tatsächlich erreichbare Leistung zu beziehen.

Fragen an den Betreuer:

- Soll der Schwerpunkt eher auf dem *Umstellungsaufwand* oder auf der *Performance-Portabilität* liegen?
- Ist der Vergleich zweier Compiler im Rahmen der Seminararbeit gewünscht, oder reicht einer?
- Gibt es Zugang zu weiterer Hardware (z. B. AMD- oder Intel-GPU) über die Hochschule?